课程笔记

软件仓库管理：从复制目录到 Git 的版本管理之道

南京大学《生成式软件工程》第 3 讲课堂笔记

lecture-to-notes 整理

2026 年 9 月 8 日



视频作者/频道：绿导师原谅你了（jyy，南京大学蒋炎岩）
来源：Bilibili（BV1kybV6DE47）
视频时长：约 100 分钟
视频链接：`https://www.bilibili.com/video/BV1kybV6DE47/`

目录

1 什么是一个软件工程项目？ 3
1.1 本章小结 . . . 3

2 为什么需要版本管理：从优盘到 Git 3
2.1 古早时期：That's All . . . 4
2.2 复制整个目录：作业-v1、作业-v2、作业-最终版 . . . 4
2.3 今天：Git 是标准答案 . . . 5
2.4 Git 从哪里来：Linux 内核的难题 . . . 5
2.5 本章小结 . . . 6

3 Git 的基本模型：仓库在文件系统里长什么样 6
3.1 从一条命令开始：eza –git . . . 6
3.2 git init：得到一个空仓库 . . . 6
3.3 展开.git：HEAD、refs 与 objects . . . 7
3.4 把模型可视化：LearnGitBranching 与 gitv . . . 8
3.5 本章小结 . . . 8

4 第一次提交：commit message 怎么写才合格 9
4.1 一个不合格的提交 . . . 9
4.2 问题清单：四个具体的坑 . . . 9
4.3 通用规范：Conventional Commits . . . 10
4.4 多问一句：为什么、该不该 . . . 10
4.5 本章小结 . . . 11

5 分支与合并：冲突发生时会发生什么 11
5.1 两个分支，一行代码 . . . 11
5.2 冲突状态：责任回到开发者手上 . . . 11
5.3 解决结果 . . . 12
5.4 本章小结 . . . 13

6 让错误可恢复：及时 commit 与回溯 13
6.1 为什么要及时 commit . . . 13
6.2 误操作之后：先保护现场，再查 reflog . . . 14
6.3 可靠回溯的前提：能构建、能测试 . . . 14
6.4 本章小结 . . . 14

7 让协作可预期：AGENTS.md 与仓库的约定 14
7.1 AGENTS.md：写给 AI 助手的协作约定 . . . 14
7.2 进组高血压时刻：什么不该提交 . . . 15
7.3 大写还是小写：一个真实项目的选择 . . . 15
7.4 约定俗成从哪里来 . . . 16
7.5 稳定共同依赖的东西 . . . 16
7.6 Git 之外：gh 与 glab . . . 16

7.7 本章小结 17

8 agent 时代的仓库管理与学习 **17**

8.1 告别古法学习 17
8.2 愿意学懂，成了真正有价值的品质 18
8.3 agent 能力之内的项目：单线程架构 18
8.4 回到软件工程：Atomic Change 推进的快照 18
8.5 当 agent 快了 1000 倍 19
8.6 本章小结 19

9 总结与延伸 **20**

9.1 讲者的收尾 20
9.2 全讲的结构化提炼 20
9.3 可以直接照做的清单 20
9.4 开放问题与下一步 20
9.5 拓展阅读 21

1　什么是一个软件工程项目？

在讨论”仓库管理”之前，需要先明确被管理的对象是什么。本讲给出的定义非常朴素：一个软件工程项目，就是一个目录、一组文件，以及一个可交付的东西。目录里的文件混合了 intent（你想做什么）、spec（应该做成什么样）和代码（实际怎么实现）三类内容——hello.c 是代码，README 是 spec，而”我希望这个程序能跑起来”则是 intent。

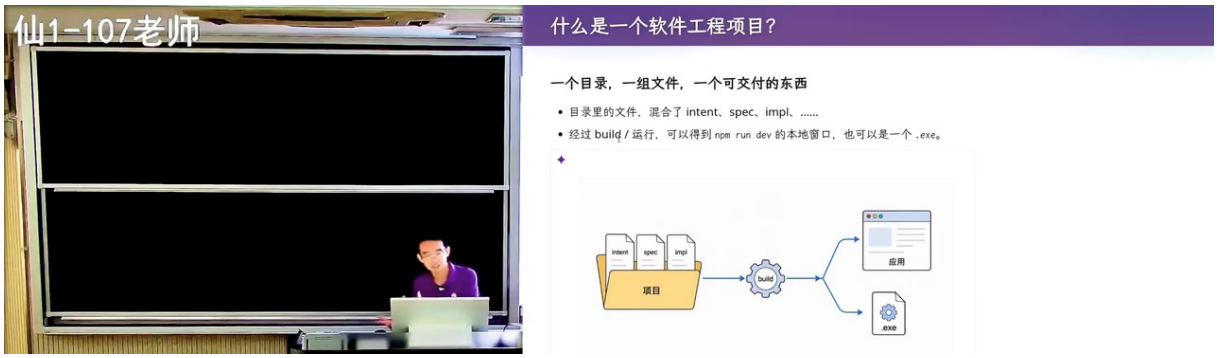


图 1：什么是一个软件工程项目：一个目录、一组文件、一个可交付的东西，目录里的文件混合了 intent、spec 与代码[1]

这个定义值得停下来想一想。讲者强调，”理应当做到”的事就是能做到的事——当你面对一个空目录时，你如何知道该在里面放什么？对多数初学者，答案来自模仿：看到老师或同学打开一个工程项目，里面有一些文件，你就在不经意之间记住了这个项目的样子；当有人向你推荐”什么什么东西发布了，你可以去看看”，你得到了一个 GitHub 或者 Git 的链接，点进去就能看到一个真实项目的结构。

本讲的核心问题

项目一旦超出”一个 hello.c 加一个编译命令”的规模，就会遇到两类问题：文件改坏了怎么找回？多人（或多 agent）同时改怎么汇合？版本管理回答的正是这两个问题。本讲从”为什么需要版本管理”讲到 Git 的基本模型，再到 agent 时代写代码时必须遵守的仓库 best practice。

因此，理解”项目 = 目录 + 文件 + 可交付物”这一朴素定义，是理解后续所有内容的前提：Git 管理的就是这个目录的历史，而 best practice 约束的就是往这个目录里放什么、不放什么。

1.1　本章小结

一个软件工程项目就是一个目录、一组文件、一个可交付的东西，其中混合着 intent、spec 与代码。项目一旦变大，”找回历史”与”汇合改动”就成为必须回答的问题——这正是版本管理的任务。下一节从没有 Git 的时代讲起，看看这个问题是怎么一步步被解决的。

2　为什么需要版本管理：从优盘到 Git

要理解 Git 解决了什么问题，最好的办法是回到没有 Git 的时代。本讲沿着”保存旧版本”这条需求线，从优盘讲到复制目录，再到 CVS、SVN，最后到达 Git。每一代工具都在回答同一个问题：怎么把旧的留下来？

[1] 视频画面时间区间：00:09:45–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

2.1 古早时期：That's All

讲者分享了他第一个 PhD student 的故事（2012 年入学）：那个年代管理项目的方式是" 一个优盘，装着全部代码"——做完一个阶段就把代码拷进优盘，项目的历史就浓缩成优盘里那一两个压缩包。这种方式被讲者称为 That's All：交付物就是全部，历史无从谈起。



图 2: 古早时期 That's All：一个优盘装着全部代码，项目历史浓缩成几个压缩包[2]

2.2 复制整个目录：作业-v1、作业-v2、作业-最终版

多数人离开优盘时代后的第一反应，是把整个目录复制一份。讲者把这称为" 最直接" 的做法：叫作业-v1、作业-v2、作业-最终版。它确实能找回旧文件，但很快会遇到瓶颈——" 哪份是从哪份改来的、谁改了什么"，要靠人记。一旦目录数量多起来，或者几个人的副本要汇合，人脑就记不住了。



图 3: Git 之前的版本管理：从复制目录、RCS 到 CVS/SVN 与 Git 的分层对比[3]

用工具记：RCS 与集中式系统

比复制目录前进一步的是让工具替你保存文件的历史版本：RCS 在单机上记录每个文件的修订史；CVS、SVN 再往前走，把历史放在一台服务器上，大家从服务器取出文件、修改、再交回去。这一步解决了" 历史存在哪里"，但历史仍然集中在一处：服务器不可用，协作就停摆。

[2] 视频画面时间区间：00:13:30–00:15:00。若为裁剪图，时间区间仍对应原视频画面。
[3] 视频画面时间区间：00:29:15–00:32:15。若为裁剪图，时间区间仍对应原视频画面。

2.3 今天：Git 是标准答案

本讲明确指出：项目版本管理在今天有了标准答案——Git。它保存当前文件之外，还能保留、比较和恢复历史版本。与其他方式相比，Git 的关键差异在于分布式：通常每个人本地就有完整历史，也记录每个版本的来路（谁改的、从哪个版本改来的），从而帮助大家汇合改动。



图 4: 今天：Git 让项目版本管理有了标准答案——保存、比较与恢复历史版本[4]

表 1: 版本管理的几种做法（据本讲内容整理）

做法	历史存哪里	来路靠谁记	关键局限
优盘（That's All）	不存在	没有人	交付物即全部
复制目录	一堆文件夹副本	靠人记	汇合时人脑记不住
RCS	本机文件	工具	单文件粒度，无协作
CVS / SVN	一台服务器	服务器	集中式，依赖单点
Git	每人本地完整历史	自动记录来路	概念多，需学习

2.4 Git 从哪里来：Linux 内核的难题

Git 不是凭空设计的。1991 到 2002 年间，Linux 内核的开发者如果想维护 Linux，主要靠邮件传递 patch 和 diff——改动怎样汇到一起，是一个真实的大项目逼出来的难题。2005 年，Git 诞生，讲者笑称" 它比我们想象的年轻一点"。一个大项目、一组真实的协作需求，最终逼出了自己的工具。



图 5: Git 从哪里来：Linux 内核 1991–2002 年的改动汇合难题[5]

[4]视频画面时间区间：00:15:15–00:16:30。若为裁剪图，时间区间仍对应原视频画面。
[5]视频画面时间区间：00:32:30–00:33:30。若为裁剪图，时间区间仍对应原视频画面。

从已有经验，长出 Git 的概念

讲者建议的学习方式是：把” 现在的你” 看成一个知识网络，学习的方式是自然地” 外推” 它。你已经知道复制目录能保存旧版本，也知道邮件传 patch 能汇合改动——Git 的概念正是从这些已有经验自然生长出来的，而不是一套需要死记的全新命令。

2.5 本章小结

版本管理的需求线是：优盘装全部代码（That’s All）→ 复制目录（靠人记来路）→ RCS/CVS/SVN（工具记，但集中在一台服务器）→ Git（每人本地完整历史、自动记录来路、帮助汇合改动）。Git 于 2005 年从 Linux 内核的真实协作难题中诞生。理解了” 为什么”，下一节进入 Git 的基本模型：一个仓库在文件系统里到底长什么样。

3 Git 的基本模型：仓库在文件系统里长什么样

知道了” 为什么”，接下来的问题是 Git 到底怎么工作。本讲的做法不是背命令，而是把真实的仓库展开给你看：从一条命令出发，一直看到.git 目录里的每一个文件。

3.1 从一条命令开始：eza –git

讲者首先演示了 eza——一个带 git 状态显示的目录列表工具。它相当于 ls 的增强版，还会” 贴心地给目录一个苹果的图标”。一条命令就能同时看到目录内容和每个项目的 git 状态，这是把” 别人项目变成学习素材” 的第一步：先看见，再理解。

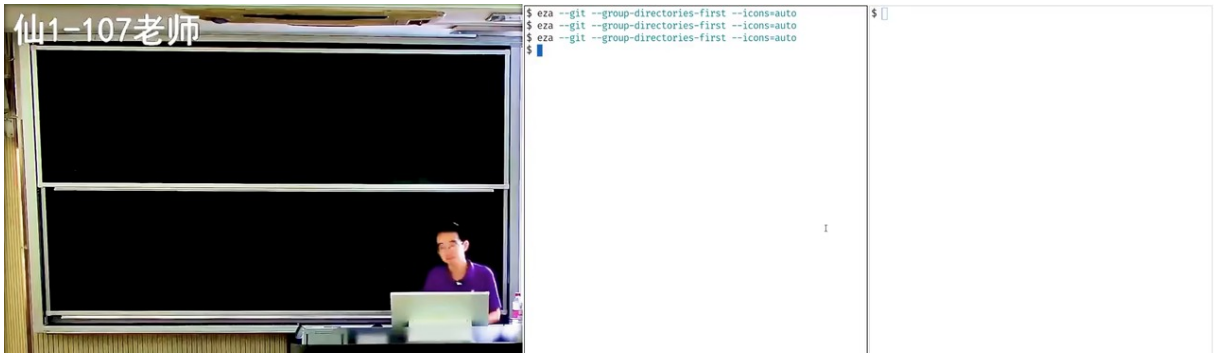


图 6: eza –git：一条命令同时查看目录内容与 git 状态[6]

3.2 git init：得到一个空仓库

开启一个项目，从 git init 开始。讲者演示了在空目录里执行它，并提醒：它会告诉你一些” 奇怪的信息”——分支叫 master（或 main），还没有任何提交。Tutorial 告诉你的基本模型是：git init 会得到一个空的 repository（仓库），后续所有历史都在这里生长。

[6] 视频画面时间区间：00:35:00–00:35:15。若为裁剪图，时间区间仍对应原视频画面。



图 7: 从空白开始：Tutorial 告诉你的基本模型——git init 得到一个空 repository[7]

```
$ git init
Initialized empty Git repository in /tmp/hello/.git/
```

Listing 1: 初始化一个空仓库（本讲演示）

3.3 展开.git：HEAD、refs 与 objects

Git 真的像 Tutorial 说的那样工作吗? 讲者把.git 目录展开在屏幕上: 里面有 HEAD、refs 和 objects。refs 下面是 heads, 记录着 main 这样的分支名;objects 则是带 hash 索引的普通文件–目录的节点、文件的节点, 甚至还有 commit 的节点。三大类型的对象由此而来：commit object 记录一次提交，tree object 记录目录结构，blob object 就是字节的序列（binary）。

三大类型对象

commit object 记录一次提交（作者、时间、指向哪个快照）；tree object 记录一层目录的结构；blob object 只管字节序列，不关心文件名。三层组合起来，一次提交就能完整描述项目的整个目录树。

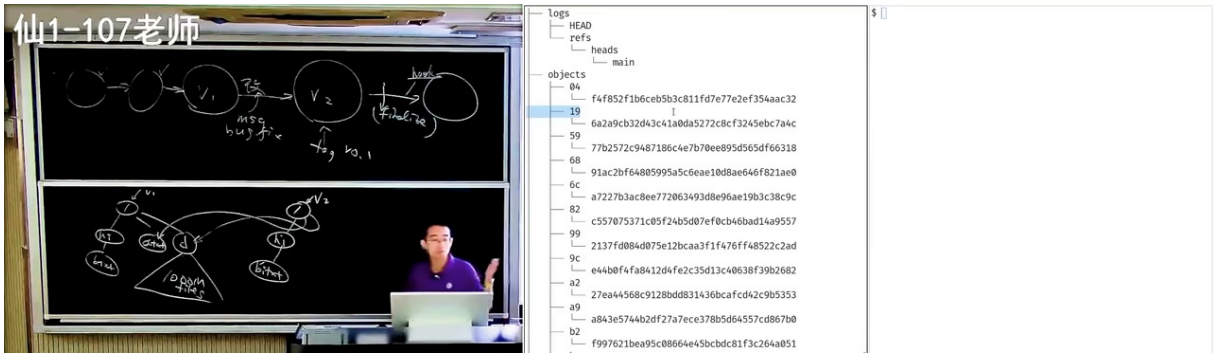


图 8: .git 目录展开：HEAD、refs 与带 hash 索引的 objects[8]

这个设计值得从第一性原理理解一遍。为什么 objects 能这样组织? 讲者给出了推理：目录本身不可能很深，没有人会去建一个一千层深的目录；每一级索引都是指数级增加的，所以很快就能把所有文件分完；你只需要创建一个非常小的增量，就得到了一个版本的快照。

[7] 视频画面时间区间：00:38:15–00:38:30。若为裁剪图，时间区间仍对应原视频画面。
[8] 视频画面时间区间：01:06:30–01:07:00。若为裁剪图，时间区间仍对应原视频画面。

从第一性原理设计 Git 是可能的

讲者明确表达了他的判断：一个大学生想要从 first principle 设计出 Git 这样的东西，是完全有可能的。理解.git 的结构不是为了背下它，而是为了确信这套机制不神秘——它就是一组带 hash 索引的普通文件，加上一个把名字指向提交的引用系统。

3.4 把模型可视化：LearnGitBranching 与 gitv

纸面上的讲解容易让人”不踏实”：commit object、tree object、blob object 真的是这样吗？讲者推荐了两层验证工具。第一层是 LearnGitBranching——一个在浏览器里模拟分支与合并的网站，它甚至模拟了各种各样的命令，git status 都有；讲者称它是”古法编程的巅峰之作之一”。

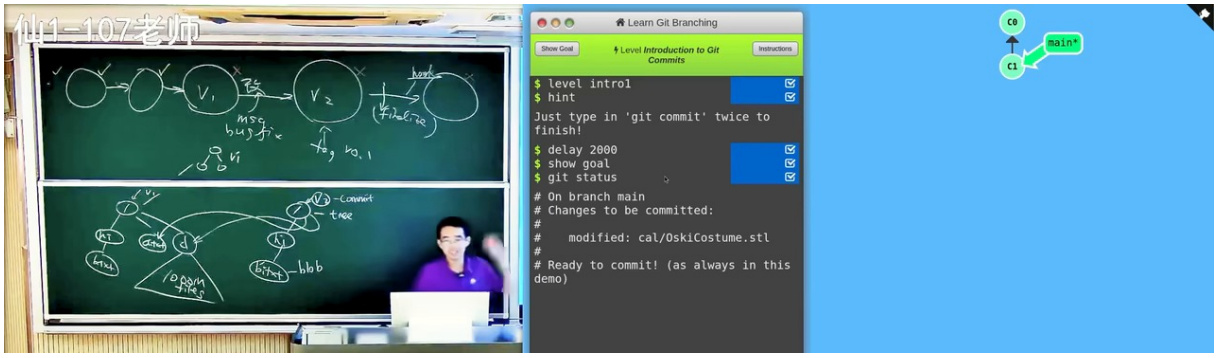


图 9: LearnGitBranching：在浏览器中可视化地模拟分支与合并[9]

第二层更彻底：讲者现场让 AI 实现了一个 gitv 命令行工具（nodejs），把一个真实的 Git 仓库彻底完整地可视化——展示任何对象之前，都从文件系统里的真实文件出发。它是用 pi 的扩展机制驱动的：pi 支持 agent_settled 这样的事件，agent 完成一轮工作后自动刷新可视化。从”纸面讲解”到”模拟器”再到”真实仓库”，三层证据互相印证。



图 10: 把真实的 Git repo 展开给你看：gitv 命令行工具[10]

3.5 本章小结

Git 的基本模型可以压缩成三句话：git init 得到一个空仓库；.git 里放着 HEAD、refs 和带 hash 索引的 objects（commit、tree、blob 三大类型）；每一次提交只记录一个非常小的增量，就构成一

[9] 视频画面时间区间：01:09:00–01:09:15。若为裁剪图，时间区间仍对应原视频画面。
[10] 视频画面时间区间：01:10:30–01:10:45。若为裁剪图，时间区间仍对应原视频画面。

个版本快照。模型可以从第一性原理推导，也可以用 LearnGitBranching 和 gitv 验证。下一节进入第一次提交：怎么写一条合格的 commit。

4 第一次提交：commit message 怎么写才合格

有了仓库，就要提交。本讲用一个真实的反例，把” 提交信息怎么写” 讲透：先看一条不合格的提交，再逐条给出问题清单，最后引出行业里通用的规范。

4.1 一个不合格的提交

讲者故意做了一条提交，信息只有一句 `I did some cool stuffs`。用 AI 追问一下就会发现：这条 commit 不符合 best practice，主要问题在提交信息，其次代码本身也有问题——提交信息的可读性有问题，” 完全看不出做了什么”。

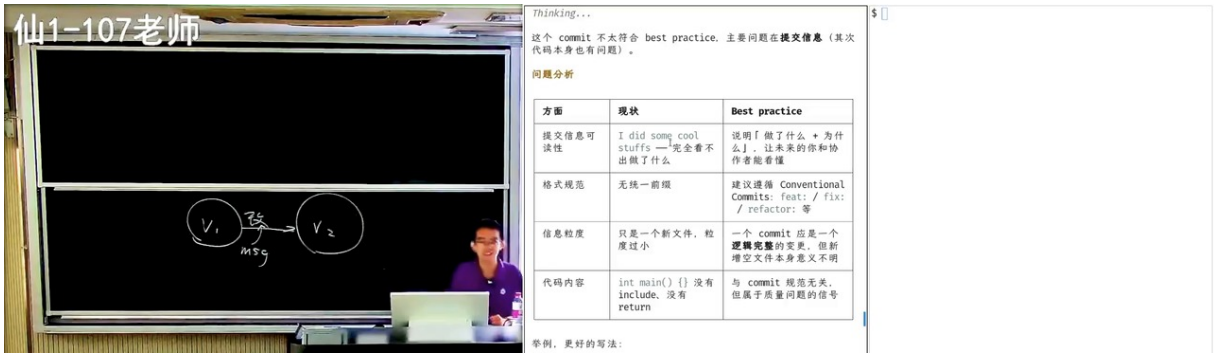


图 11: 这个 commit 不太符合 best practice：主要问题在提交信息无信息量[11]

4.2 问题清单：四个具体的坑

把这条提交展开，讲者列出了四个按严重程度排序的问题。第一，提交信息毫无信息量（slop）——没说明改了什么；空提交更特殊，提交信息本身在说” 这是个空提交”，但 commit message 应当描述内容而不是状态。第二，提交粒度与内容混杂——两行改动混在一起，修 bug 的提交和” 顺手格式化” 应该分开。第三，前缀如 feat:、fix: 也值得考虑。第四，没有关联到上下文——若仓库有 issue 跟踪，建议引用（Closes #N），无说明文字的提交让别人无法追溯。

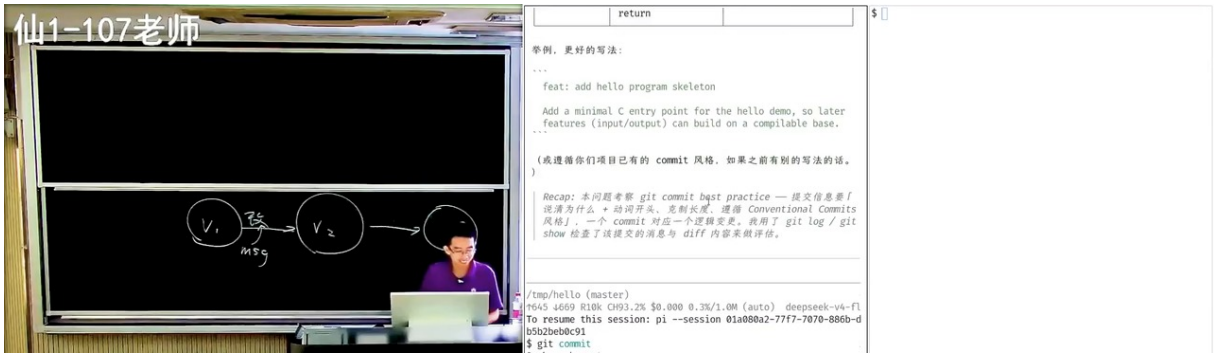


图 12: 更好的写法：feat: 前缀与祈使句式的描述[12]

[11] 视频画面时间区间：00:39:30–00:41:00。若为裁剪图，时间区间仍对应原视频画面。

```
$ git commit -m "feat: add hello program skeleton

Add a minimal hello program that prints a greeting,
so the repository has a first deliverable."
```

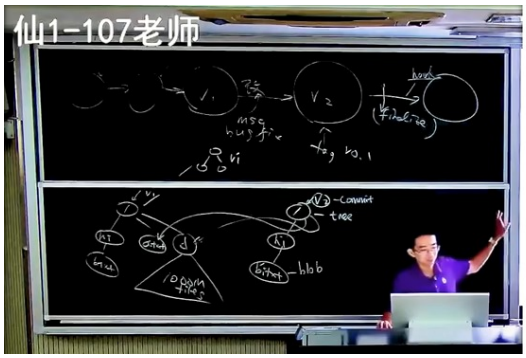
Listing 2: 更好的提交写法（据本讲示例整理）

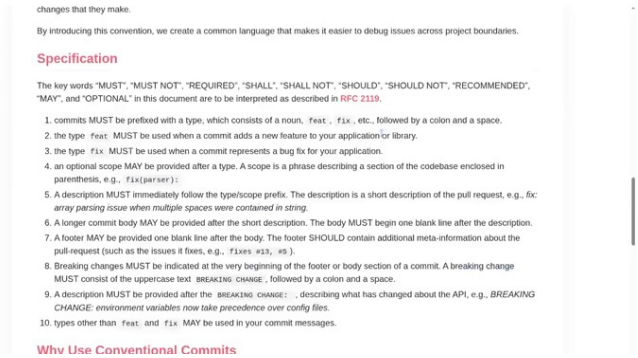
空提交与重写

Git 也允许你在不改任何代码的时候提交：`git commit --allow-empty` 会创建一个空提交。但重写历史时应把空提交剔除、不再存在；二进制文件（如编译出的 hello）也不该进入历史——新历史从未包含它，或用.gitignore 让它根本不入库。

4.3 通用规范：Conventional Commits

这些约定不止适用于一个项目。行业里有一套通用的 commit message 规范——Conventional Commits：用 feat:、fix:、chore: 等前缀说明提交类型，用祈使句描述内容。讲者引用了规范文档里的一句话：By introducing this convention, we create a common language——引入这套约定，我们就创造了一种共同语言。review、revert、bisect 这些操作才有可靠的单位。




changes that they make.

By introducing this convention, we create a common language that makes it easier to debug issues across project boundaries.

Specification

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119.

1. commits MUST be prefixed with a type, which consists of a noun, `feat`, `fix`, etc., followed by a colon and a space.
2. the type `feat` MUST be used when a commit adds a new feature to your application or library.
3. the type `fix` MUST be used when a commit represents a bug fix for your application.
4. an optional scope MAY be provided after a type. A scope is a phrase describing a section of the codebase enclosed in parenthesis, e.g., `fix(parser):`
5. A description MUST immediately follow the type/scope prefix. The description is a short description of the pull request, e.g., *fix: array parsing issue when multiple spaces were contained in string.*
6. A longer commit body MAY be provided after the short description. The body MUST begin one blank line after the description.
7. A footer MAY be provided one blank line after the body. The footer SHOULD contain additional meta-information about the pull-request (such as the issues it fixes, e.g., `fixes #13, #5`).
8. Breaking changes MUST be indicated at the very beginning of the footer or body section of a commit. A breaking change MUST consist of the uppercase text `BREAKING CHANGE`, followed by a colon and a space.
9. A description MUST be provided after the `BREAKING CHANGE: `, describing what has changed about the API, e.g., *BREAKING CHANGE: environment variables now take precedence over config files.*
10. types other than `feat` and `fix` MAY be used in your commit messages.

Why Use Conventional Commits

图 13: Commit Message 规范与 Conventional Commits：创造一种共同语言[13]

4.4 多问一句：为什么、该不该

commit 规范背后还有一层更基本的要求：哪怕只是放一个 hello.c，缺少足够的知识也会犯错误。讲者演示了 agent 时代的一个具体场景：agent 改了代码后，多问一句” 这个改动该不该提交”，让它先验证能否编译——`gcc -Wall hello.c -o /tmp/hello_test`，编译器立刻报出 `error: 'a' undeclared`（a 没被声明）和 `error: expected ';'`（语句缺分号）。

```
$ gcc -Wall hello.c -o /tmp/hello_test 2>&1
hello.c: error: 'a' undeclared      # a 没被声明
hello.c: error: expected ';'        # 语句缺分号
```

Listing 3: agent 修改后的验证（本讲演示）

[12]视频画面时间区间：00:41:15–00:41:30。若为裁剪图，时间区间仍对应原视频画面。

[13]视频画面时间区间：01:28:45–01:29:00。若为裁剪图，时间区间仍对应原视频画面。

每一次提交的代码应该能够通过构建和测试——讲者强调这是一个非常基础的要求。在你做任何 git 操作之前，都要逐个检查文件的变化；agent 说” 测试都通过” 之后，你也要看一眼这个东西看起来是不是真的 OK。

4.5 本章小结

合格的提交有四个要素：信息量（说清改了什么）、粒度（修 bug 与格式化分开）、前缀（feat:/fix: 表明类型）、上下文（引用 issue）。Conventional Commits 把这些约定变成行业的共同语言。再往深处走一层：每次提交都应能通过构建和测试，这是 review 与 revert 可靠的前提。下一节看多个提交汇合时会遇到什么——分支与冲突。

5 分支与合并：冲突发生时会发生什么

单人提交只用到 Git 的一半能力。Git 的另一半是分支与合并——这也是” 帮助大家汇合改动” 的关键。本讲用两个分支改同一行代码的例子，把冲突从触发到解决完整走了一遍。

5.1 两个分支，一行代码

实验设计很简单：分支 A 把 hello.c 的首行改成 `int main(void){` 并提交（94d2b56）；分支 B 把同一行改成 `int main(int argc, char **argv){`。两份改动都合法，但改的是同一个位置——合并时 Git 无法替你决定用哪一个。

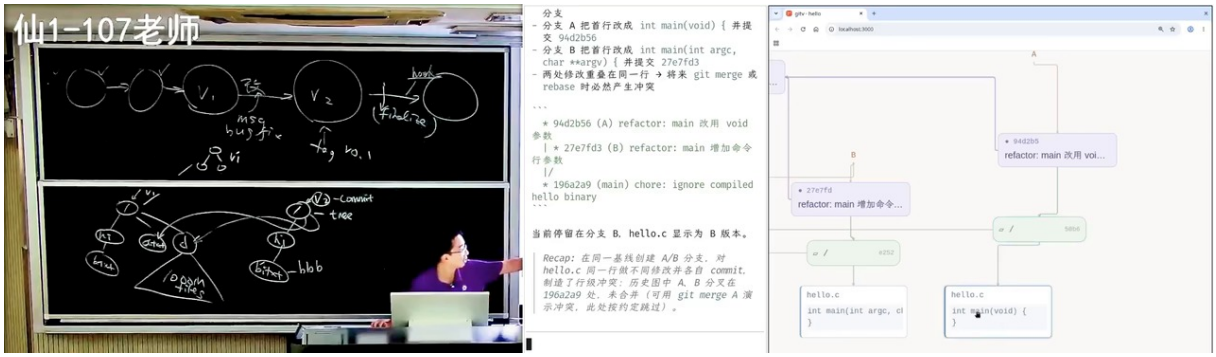


图 14: 分支 A 与分支 B 各自修改首行并提交[14]

5.2 冲突状态：责任回到开发者手上

合并被触发后，Git 停在未完成合并的状态。此时文件里会出现” 好多等号、好多箭头” 的冲突标记，分成 base（共同祖先）、ours（当前分支）、theirs（对方分支）三段。讲者在讲保研面试观察时就强调过这一点：冲突以后，不是立即就被 resolve 了，而是这个责任到了每一个开发者手上——机器不知道你想要哪个，你必须自己决定。

[14] 视频画面时间区间：01:15:45–01:16:00。若为裁剪图，时间区间仍对应原视频画面。

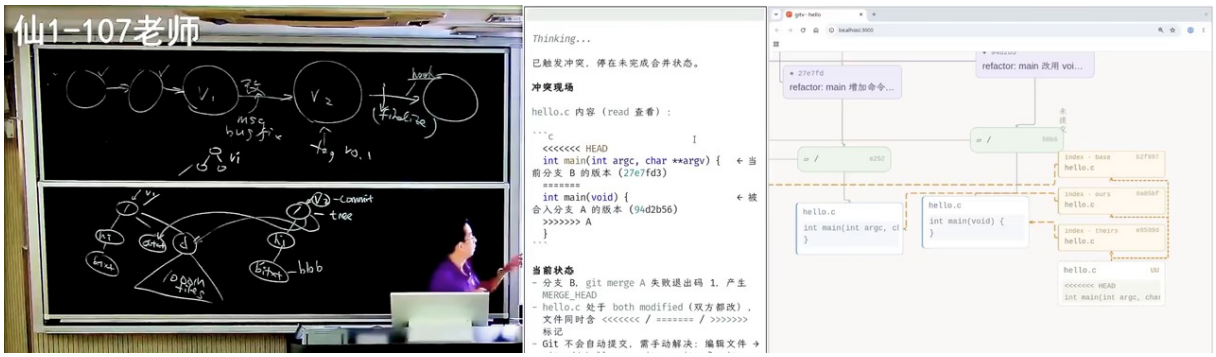


图 15: 合并触发冲突，停在未完成合并状态[15]

冲突时先停下来看一眼

agent 会说“来吧”，然后自动把冲突修好、测试全绿。但冲突标记出现的那一刻，正是学习的机会：看一看 base、ours、theirs 各自改了什么。如果你愿意在冲突的时候停下来，你会看到一个非常神奇的东西；直接跳过，下次遇到真正的冲突还是会慌。

5.3 解决结果

手工选择保留哪一侧（或融合两侧）之后，冲突标记被移除，临时文件消失，合并完成。本讲的例子里，最终保留的是参数更完整的一侧：`int main(int argc, char **argv){`。

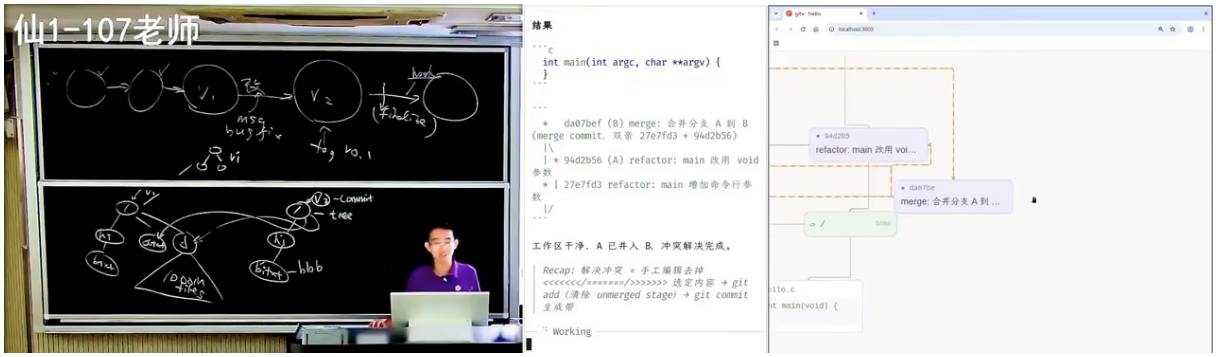


图 16: 冲突解决结果：保留 int main(int argc, char **argv){}[16]

```
<<<<<<< HEAD
int main(int argc, char **argv) {
=======
int main(void) {
>>>>>>> 94d2b56
```

Listing 4: 冲突标记的典型形态（据本讲演示整理）

[15] 视频画面时间区间：01:16:30–01:16:45。若为裁剪图，时间区间仍对应原视频画面。

[16] 视频画面时间区间：01:17:30–01:17:45。若为裁剪图，时间区间仍对应原视频画面。

用 gitv 看清历史

冲突解决后，历史长什么样？讲者用 gitv 展示了递归调用的调用树（如汉诺塔 hanoi 的多层递归），每一层展开都对应真实的文件系统状态。看到这些，你就明白 Git 记录的不是”文件的最后样子”，而是一棵可以随时回溯的快照树。

5.4 本章小结

两个分支改同一行，合并时 Git 会停在未完成合并状态，把决定权交还开发者：base、ours、theirs 三段标记必须手工处理，责任在每一个开发者身上。解决之后，历史汇合成一棵可回溯的快照树。下一节看 Git 最实用的能力：当你改错了东西，怎么把历史找回来。

6 让错误可恢复：及时 commit 与回溯

人和 Agent 都会犯错误：设计会被改掉，对的代码会被改错，哪怕只是手滑。本讲给”回溯”定的目标不是”恢复某个文件”，而是项目级的回溯时间——在任何时刻，都能回到一个可构建、可测试的快照。

6.1 为什么要及时 commit

讲者给出了明确的因果链：在复杂的项目里，随着你修改的东西越多，你错的东西就越可能多。小项目刚开始的时候其实不要紧，一次提交个两三千行（2000–3000 行）代码都 OK；但一旦项目里有了错综复杂的关系，一个大 commit 可能会 break 项目里多处的东西。所以 Best Practice 是：实验前留检查点，及时 commit。



图 17: Best Practices：让错误可恢复——及时 commit、备份与恢复[17]

工作区里的修改，Git 未必能救

检查点必须先提交，Git 才能救你：只存在于工作区里的修改（没 add、没 commit），Git 未必能恢复。同理，远端 clone 不包含未提交文件——别把”推到远端”当成未提交工作的备份。

[17] 视频画面时间区间：01:30:00–01:31:00。若为裁剪图，时间区间仍对应原视频画面。

6.2 误操作之后：先保护现场，再查 reflog

真的改错了怎么办？讲者给的操作顺序是：误操作后先保护现场，再查 reflog。reflog 记录了 HEAD 的每一次移动，即使提交被 reset 掉，也能从这里找回。找到了错误引入的点，就可以 review、revert，或者用 bisect 二分查找——讲者把它比喻成“在一个后缀都是 0/1 的数组里找第一个 1”：每次取中间点测试，把坏提交的范围折半缩小。

bisect：二分查找坏提交

把提交历史想象成一个很长的数组，每项是 0（好）或 1（坏），现在在最新版本上发现一个 bug——坏标记从某处开始全变成 1。bisect 每次取中间的提交测试：好就往后找，坏就往前找，几步之内就把“第一个 1”锁定了。前提仍然是每个提交都能构建、能测试。

6.3 可靠回溯的前提：能构建、能测试

reflog 和 bisect 之所以可靠，前提是每一个检查点都是合格的单位。讲者在”让探索可检验”里给出的规则是：发布前按一个意图拆分提交，把修复和相应的测试放在一起，顺手格式化另开提交；整理后的提交能构建、能测试，review、revert、bisect 才有可靠的单位。这也回应了前一章的结论：agent 声称”测试都通过”之后，你仍要检查每一处变化。

6.4 本章小结

让错误可恢复的机制链是：及时 commit 留检查点 → 误操作后先保护现场再查 reflog → 用 review/revert/bisect 定位并回退。前提是每次提交都能构建、能测试，否则回溯就失去了可靠的单位。下一节从”自己用”转向”和别人协作”：仓库的约定怎么让协作可预期。

7 让协作可预期：AGENTS.md 与仓库的约定

Git 的另一个角色是协作载体：当你把仓库交给别人（或交给 AI 助手）时，仓库里的约定决定了对方会不会把事情搞砸。本讲从一份特殊的文件讲起。

7.1 AGENTS.md：写给 AI 助手的协作约定

本讲的例子里出现了一个未跟踪的 AGENTS.md。它的内容是一段写给协作方（比如 AI 助手）的约定：哪些文件不应提交、临时文件放在哪里。讲者的判断是：AGENTS.md 可以提交，这符合 best practice——它是仓库的一部分，让每个进入仓库的协作者（人或 agent）先读到规则再动手。

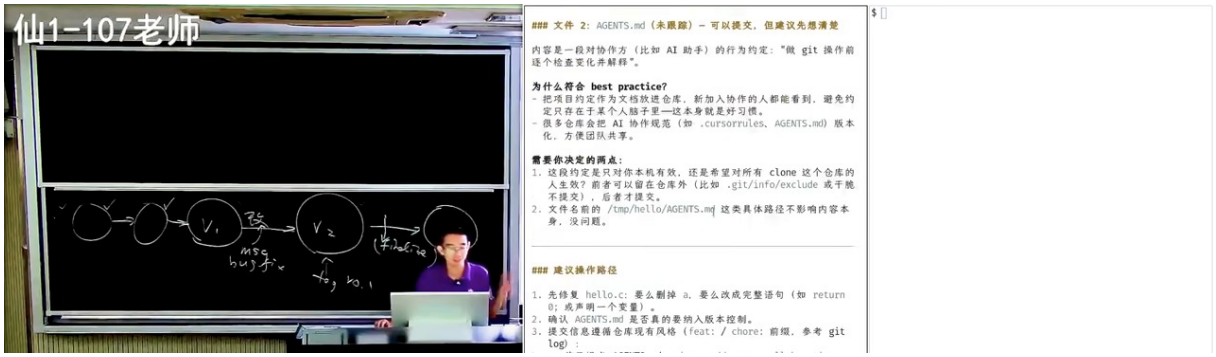


图 18: AGENTS.md：写给 AI 助手的协作约定文件[18]

7.2 进组高血压时刻：什么不该提交

讲者回忆了和许多研究生新生说过的”高血压时刻”：生成的文件不提交（dist/、log 等），个人的配置不提交。这些内容不是项目的一部分，把它们提交进历史，会把 review 和回溯的信号全部污染。

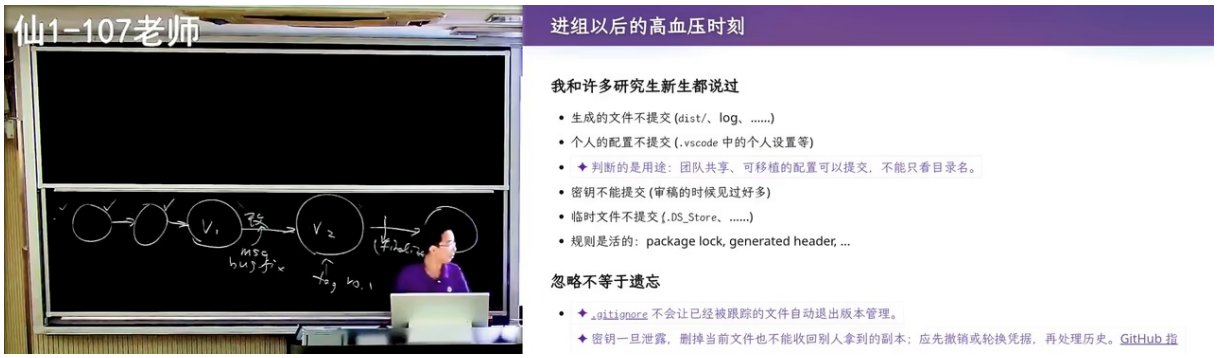


图 19: 进组高血压时刻：生成文件与个人配置不提交[19]

临时文件与.gitignore

默认带点（.）开头的文件会被.gitignore 挡住，但临时文件还是可能被提交进来。规则并不总是机械的：不是所有 generated file 都不该提交（例如.github/workflows 这类约定俗成的东西就应该提交）。判断标准是它是否属于项目本身。

7.3 大写还是小写：一个真实项目的选择

仓库的检查规则也有自己的命名问题。本讲展示了一个真实项目里的选择：一条检查规则叫`[TYPO-CHECK]Docs should be typo-free`，方括号里的规则名该大写还是小写？讲者的直觉是”大写似乎更好，但不是极强”，一开始不太说得出原因。想清楚之后他选择了 TYPO-CHECK：用大写字形提示不同的角色——方括号里的字符是固定的规则标识，是一句可以被引用的规则名字，而不是句子的一部分。

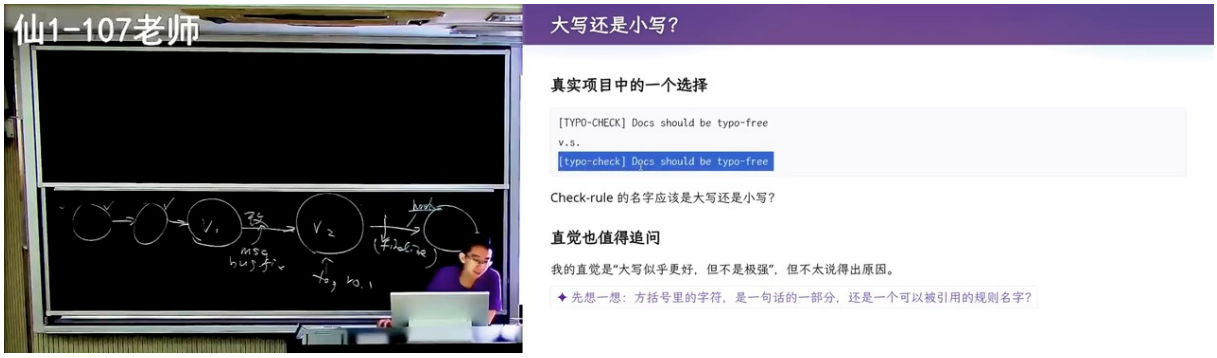


图 20: 大写还是小写：TYPO-CHECK 用字形提示不同角色[20]

[18] 视频画面时间区间：00:55:00–00:55:15。若为裁剪图，时间区间仍对应原视频画面。
[19] 视频画面时间区间：00:57:30–00:58:45。若为裁剪图，时间区间仍对应原视频画面。
[20] 视频画面时间区间：01:00:45–01:01:00。若为裁剪图，时间区间仍对应原视频画面。

这类规则可以挂在 git 的 pre-commit hook 上做检查——讲者提到他有几十个这样的 checking rules，而且不同项目的工程标准不同，同一个规则在不同仓库里的严格程度也可以不一样。

7.4 约定俗成从哪里来

这些规则有一个共同特点：它们不是某个人发明的，而是软件和系统里约定俗成的 best practice。讲者提醒：而每一个约定背后都是血的教训，或者对旧设计的修复。所以遇到 best practice 时，直觉值得追问——追问得出原因，你就学到了设计；追问不出，至少也记住了边界。



图 21：约定俗成的 best practice：每一个都是血的教训，或对旧设计的修复[21]

7.5 稳定共同依赖的东西

协作 best practice 的第三条是：不擅自改写别人依赖的历史。改写（rewrite）会把整个依赖链破坏掉——别人正在基于你发布的某个版本工作，你把历史重写了，对方的基线就悬空了。这与”让错误可恢复”形成对照：自己的历史可以整理，别人依赖的历史要稳定。

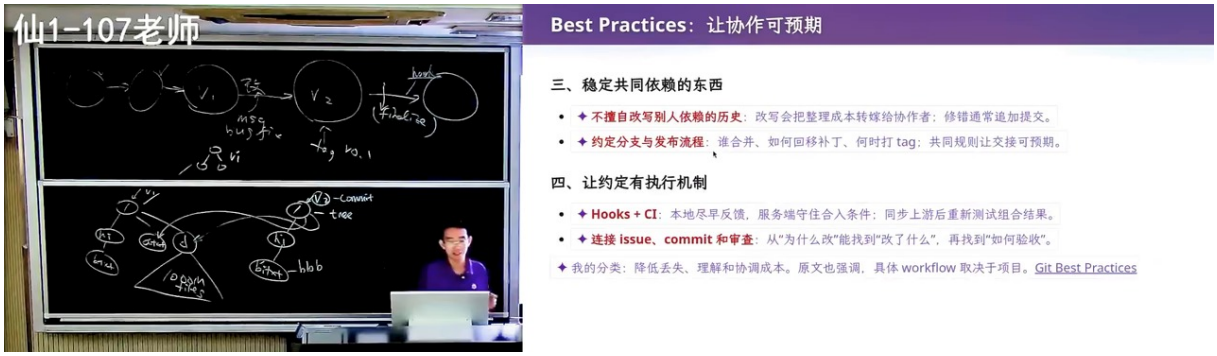


图 22：Best Practices：让协作可预期——稳定共同依赖的东西[22]

7.6 Git 之外：gh 与 glab

从问题到合入，还有一段流程在 Git 之外：提 issue、发 pull request、跑 CI。GitHub 用 gh 把这段流程接到命令行，GitLab 对应有 glab；甚至为了管理 non-git 的部分，也专门有工具。讲者强调的仍是那句话：你想要的就一定有人做过——先找工具，再动手造。

[21] 视频画面时间区间：01:03:00–01:03:15。若为裁剪图，时间区间仍对应原视频画面。
[22] 视频画面时间区间：01:31:15–01:31:30。若为裁剪图，时间区间仍对应原视频画面。

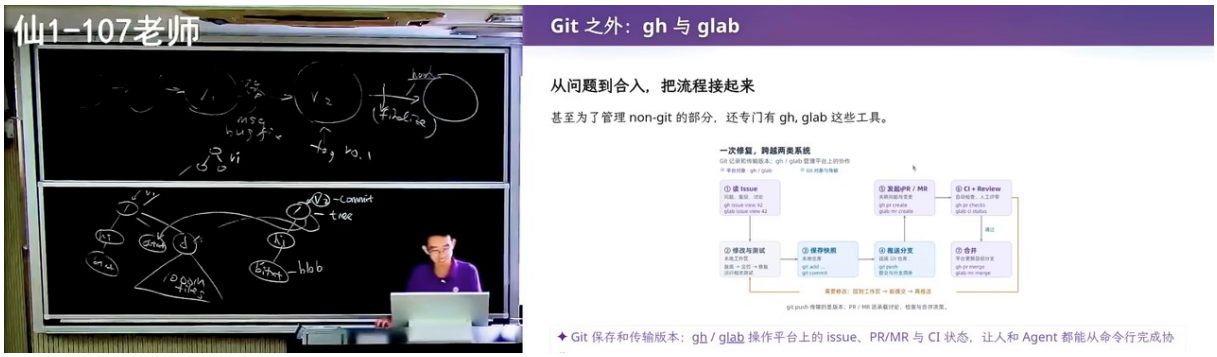


图 23: Git 之外：gh 与 glab 把” 问题到合入” 的流程接起来[23]

7.7 本章小结

让协作可预期的约定链是：AGENTS.md 写明协作规则 → 生成文件与个人配置不提交 → 检查规则有清晰的名字（TYPO-CHECK 用大写字形标记角色）→ 不擅自改写别人依赖的历史 → 用 gh/glab 把 Git 之外流程接起来。每一条约定背后都是血的教训或对旧设计的修复。下一节回到 agent 时代：这些实践怎么和 AI 一起工作。

8 agent 时代的仓库管理与学习

本讲反复强调时间背景：现在是 agent 时代，你们都是 agent native 的人——有了十倍到一百倍的生产力提升。这一节把前面的仓库实践放到这个背景下，看讲者的观察、判断和留到下节课的问题。

8.1 告别古法学习

agent 时代改变了学习的路径。讲者把”git status、git add、git commit” 这样的命令清单称为” 古法”：agent native 的人不必从命令背起，而是根据计算机科学定律——你想要的东西就一定有人做过——先找现成的工具和项目，把关键细节以直观的方式呈现出来。前面推荐的 LearnGitBranching、eza、gitv，都是这条路径的实例。

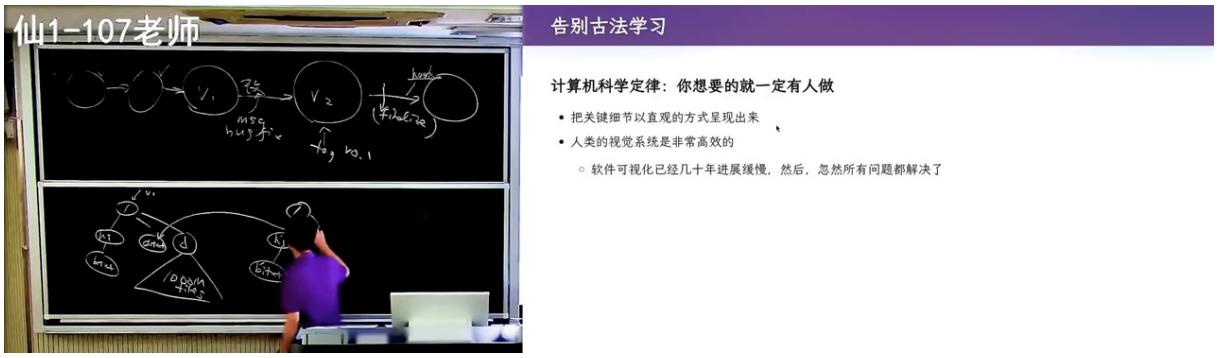


图 24: 告别古法学习：计算机科学定律——你想要的就一定有人做[24]

[23] 视频画面时间区间：01:33:15–01:33:30。若为裁剪图，时间区间仍对应原视频画面。
[24] 视频画面时间区间：01:05:15–01:05:30。若为裁剪图，时间区间仍对应原视频画面。

8.2 愿意学懂，成了真正有价值的品质

工具越强，人的那部分就越清楚。讲者观察到：把” 细节做好” 的门槛在不断提高——今天你们真正有价值的品质，就是你们愿意去花一点时间学懂；只要有人学会了，招人的单位就会用这个标准来衡量你。所以” 愿意学懂” 不再是情怀，而是可衡量的品质。

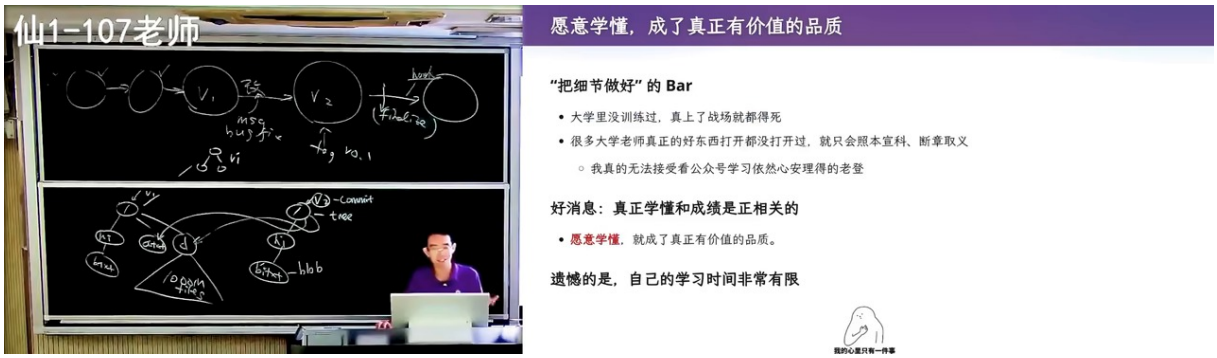


图 25: 愿意学懂成了真正有价值的品质[25]

8.3 agent 能力之内的项目：单线程架构

讲者用一个真实项目（Pico Playground）展示了 agent 时代项目该怎么做。工作方式是” 架构设计单线程，反复讨论”：和高智力模型讨论架构，然后让 agent 落地；就几万个 token 就可以做出一个 playground，你可以肉眼看着它不断变好。架构上有两条纪律：一是单线程推进，避免多份并行改动互相打架；二是 libs/ 里是项目重要的抽象，接口和实现严格分离，设计应当满足” 只阅读 core.ts 就能懂”。

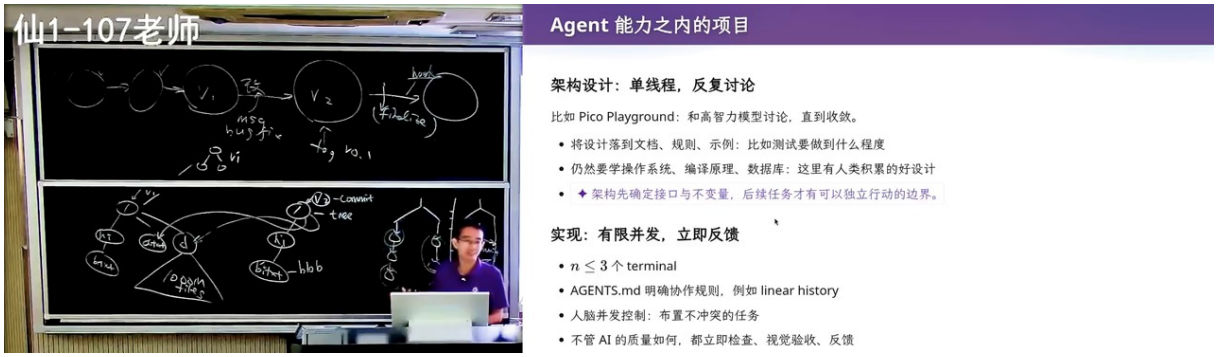


图 26: Agent 能力之内的项目：架构设计单线程，反复讨论[26]

8.4 回到软件工程：Atomic Change 推进的快照

讲了一段 Git 之后，讲者把镜头拉回来：其实这门课讲的还是软件工程。项目的本质被概括成一句话——项目是由 Atomic Change 推进的快照：可以分支，可以合并，这是人类成功的经验。前面所有的 Git 实践（规范提交、合理拆分、稳定依赖）都在为这句话服务。

[25]视频画面时间区间：01:24:45–01:25:00。若为裁剪图，时间区间仍对应原视频画面。
[26]视频画面时间区间：01:39:45–01:40:00。若为裁剪图，时间区间仍对应原视频画面。

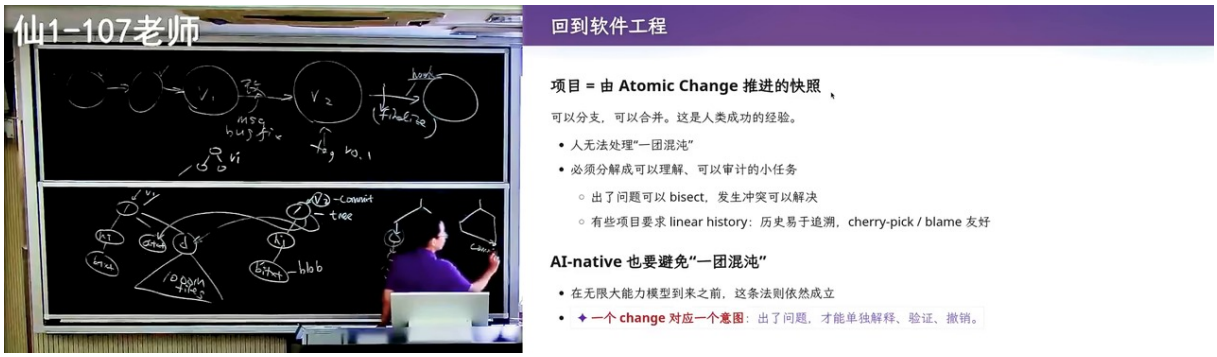


图 27: 回到软件工程：项目是由 Atomic Change 推进的快照[27]

8.5 当 agent 快了 1000 倍

本讲留下了一个开放问题。讲者称之为 "非常可怕的事情"：如果 agent 快了一千倍，代码产能暴涨——agent 可以 1000 倍更快地生产代码——我们还怎么样管理版本控制？

讲者的判断与本讲的边界

讲者非常确信 agent 能力会不断增长，越来越多的项目能够被 agent 以轻量讨论加自动落地的方式完成；但 agent 快 1000 倍时版本控制怎么管理，他留到下一节课讨论。本讲给出的是这个问题的地基：无论 agent 多快，Atomic Change 的快照模型、合格的提交单位、稳定的依赖，仍然是版本控制的底座。



图 28: 当 Agent 快了 1000 倍：代码产能暴涨的思想实验[28]

8.6 本章小结

agent 时代改变的是学习路径（告别古法、先找现成工具）和价值标准（愿意学懂成为可衡量的品质），而不是版本控制的底层逻辑：项目仍是由 Atomic Change 推进的快照，agent 项目仍需要单线程架构和严格的接口分离。当 agent 快 1000 倍时版本控制怎么管理，是留给下一讲的问题。最后一节综合全讲，给出可执行的建议。

[27] 视频画面时间区间：01:33:30–01:36:15。若为裁剪图，时间区间仍对应原视频画面。
[28] 视频画面时间区间：01:40:00–01:40:15。若为裁剪图，时间区间仍对应原视频画面。

9 总结与延伸

9.1 讲者的收尾

本讲结尾，讲者表达了两个判断。其一，Pico Playground 这样的项目是 agent 能力之内的项目，而且他非常确信 agent 能力会不断增长，越来越多的项目能够被 agent 以" 轻量讨论加自动落地" 的方式完成。其二，agent 快了 1000 倍之后版本控制怎么管理，是一个真实的问题——他没有在本讲给出答案，而是留到下一节课讨论。

9.2 全讲的结构化提炼

把全讲压缩成一条主线：软件工程项目是" 目录 + 文件 + 可交付物"，它的两个固有问题是找回历史和汇合改动；Git 用" 每人本地完整历史 + 自动记录来路" 解决这两个问题；而让 Git 真正可用的，是一组约定俗成的 best practice。各章的结论可以汇成下面的清单：

- **版本管理的需求线**：优盘（That's All）→ 复制目录（靠人记）→ RCS/CVS/SVN（集中式）→ Git（分布式，本地完整历史）；
- **基本模型**：.git 里是 HEAD、refs 和带 hash 索引的 objects（commit/tree/blob 三大类型），每次提交只记录小的增量；
- **合格的提交**：信息量、粒度、前缀（feat:/fix:）、上下文（issue 引用），且每次提交能构建、能测试；
- **冲突**：合并停在中途，base/ours/theirs 的决定权在开发者手上；
- **可恢复**：及时 commit 留检查点，误操作先保护现场再查 reflog，用 review/revert/bisect 回退；
- **协作**：AGENTS.md 写明规则，生成文件与个人配置不提交，规则名用大写字形标记角色，不擅自改写别人依赖的历史；
- **agent 时代**：告别古法学习，先找现成工具；愿意学懂是可衡量的品质；项目由 Atomic Change 推进。

9.3 可以直接照做的清单

对初次接触 Git 的读者，本讲的内容可以落成一组具体动作：新项目先 `git init`；每次实验前 commit 一个检查点；提交信息按 Conventional Commits 写，修 bug 与格式化分开；生成文件与个人配置写进.gitignore；改错先 `git reflog` 再决定回退方式；把 AGENTS.md 提交进仓库，让 AI 助手先读规则再动手。

9.4 开放问题与下一步

本讲留下的问题是：当 agent 快了 1000 倍、代码产能暴涨时，版本控制还怎么管理？讲者会在下一节课讨论。对本讲义读者而言，自然的下一步是把本章的清单在一个真实项目上走一遍：初始化仓库、做出第一次规范的提交、故意制造一次冲突并解决它、用 reflog 找回一次误操作——每一步都对应本讲的一个教学原子。

9.5 拓展阅读

- 视频原文：BV1kybV6DE47（软件仓库管理，NJU 生成式软件工程第 3 讲）；
- LearnGitBranching：讲者推荐的分支可视化学习网站（本讲称其为”古法编程的巅峰之作之一”）；
- Conventional Commits：本讲引用的行业 commit message 规范（”By introducing this convention, we create a common language”）；
- 《Pro Git》：本讲 slide 中引用的参考书。